10 preguntas de concurrencia para tu próxima entrevista iOS

Código real y la salida real. Cada respuesta se compiló y se corrió con Swift 6.4 y Xcode 27.0.

Nico Valentini · nicovma.github.io

Cómo usar esto

Cada pregunta trae código y una consigna: qué imprime, o si compila. Contestá en voz alta antes de pasar a las respuestas, como si estuvieras en la entrevista. Están todas juntas al final.

Mientras armaba esto encontré errores en respuestas que tenía por buenas. Una decía que `main.sync` se cuelga, cuando en realidad crashea. Otra, que un actor funciona como una cola serial, y no hacen lo mismo. Por eso cada salida de acá es la que dio la ejecución, no la que debería dar.

1. Data(contentsOf:) en viewDidLoad
2. El callback de URLSession
3. Task {} dentro de @MainActor
4. await en un ViewModel @MainActor
5. Actor contra cola serial
6. Dos retiros en paralelo
7. Una continuation mal cerrada
8. sync sobre colas seriales
9. Un mock Sendable
10. Un ViewModel que es actor

Pregunta 1 mid

Data(contentsOf:) en viewDidLoad

El main thread tiene que atender la UI cada 100 ms (los "latidos"). A los ~150 ms corre `viewDidLoad()`, que baja datos con `Data(contentsOf:)` de un servidor que tarda 1 segundo. ¿Qué imprime?

```
let url = URL(string: "http://127.0.0.1:8765/perfil")!   // responde en 1 s

func latido(_ n: Int) {
    print("latido \(n) a ~\(ms()) ms")
    if n < 6 { DispatchQueue.main.asyncAfter(deadline: .now() + 0.1) { latido(n + 1) } }
else { exit(0) }
}

func viewDidLoad() {
    print("viewDidLoad: pido datos a ~\(ms()) ms")
    let data = try? Data(contentsOf: url)
    print("viewDidLoad: llegaron \(data?.count ?? 0) bytes a ~\(ms()) ms")
}

DispatchQueue.main.async { latido(1) }
DispatchQueue.main.asyncAfter(deadline: .now() + 0.15) { viewDidLoad() }
dispatchMain()
```

Entorno: Swift 6.4 · Xcode 27.0 · modo Swift 6 · macOS 26

Pregunta 2 mid

El callback de URLSession

¿Qué imprime? ¿Qué bug tiene la línea comentada?

```
print("antes de resume, main thread: \(Thread.isMainThread)")
URLSession.shared.dataTask(with: url) { data, _, _ in
    print("en el callback, main thread: \(Thread.isMainThread)")
    // self.nombreLabel.text = String(decoding: data!, as: UTF8.self)
}.resume()
```

Entorno: Swift 6.4 · Xcode 27.0 · modos Swift 6 y 5 · macOS 26 (AppKit)

Pregunta 3 senior

Task {} dentro de @MainActor

¿Qué imprime cada versión? ¿`cargarPantalla()` espera al `await` de adentro antes de retornar?

```
@MainActor
func cargarPantalla() {
    print("1")
    Task {
        print("2")
        try? await Task.sleep(for: .milliseconds(50))
        print("3")
    }
    print("4")
}

@MainActor
func cargarPantallaInmediata() {
    print("1")
    Task.immediate {          // Swift 6.2, SE-0472
        print("2")
        try? await Task.sleep(for: .milliseconds(50))
        print("3")
    }
    print("4")
}
```

Entorno: Swift 6.4 · Xcode 27.0 · modo Swift 6 · macOS 26 · Task.immediate requiere Swift 6.2 y runtime iOS/macOS 26

Pregunta 4 senior

await en un ViewModel @MainActor

`PantallaViewModel` es `@MainActor`. Mientras `fetch()` espera 250 ms, ¿la "UI" (un tick en el main actor cada 100 ms) sigue respondiendo? ¿Y si `fetch()` hace 250 ms de trabajo sincrónico antes de su primer `await`?

```
@MainActor
final class PantallaViewModel {
    func fetch() async {
        print("fetch: arranca")
        try? await Task.sleep(for: .milliseconds(250))   // simula la red
        print("fetch: termina")
    }

    func fetchConTrabajoPesado() async {
        print("fetch: arranca")
        let fin = Date().addingTimeInterval(0.25)
        while Date() < fin {}                             // trabajo síncrono
        try? await Task.sleep(for: .milliseconds(1))
        print("fetch: termina")
    }

    func onAppear(pesado: Bool) {
        Task { pesado ? await fetchConTrabajoPesado() : await fetch() }
    }
}

@MainActor
func simularUI() async {
    for i in 1...4 {
        print("UI tick \(i) (~\(ms) ms)")    // ms desde el arranque, redondeado a 50
        try? await Task.sleep(for: .milliseconds(100))
    }
}
```

Entorno: Swift 6.4 · Xcode 27.0 · modo Swift 6 · macOS 26

Pregunta 5 senior

Actor contra cola serial

Las dos versiones leen un contador, esperan 50 ms y escriben `leído + 1`. Se llaman dos veces "a la vez" (A y B). ¿Qué imprime cada una? ¿Cuánto vale el contador al final?

```
actor ContadorActor {
    private var valor = 0

    func sumarLento(_ id: String) async {
        print("actor \(id): lee \(valor)")
        let leido = valor
        try? await Task.sleep(for: .milliseconds(50))   // punto de suspensión
        valor = leido + 1
        print("actor \(id): escribe \(valor)")
    }
}

final class ContadorCola: @unchecked Sendable {
    private let cola = DispatchQueue(label: "contador.serial")
    private var valor = 0

    func sumarLento(_ id: String) {
        cola.async {
            print("cola \(id): lee \(self.valor)")
            let leido = self.valor
            Thread.sleep(forTimeInterval: 0.05)
            self.valor = leido + 1
            print("cola \(id): escribe \(self.valor)")
        }
    }
}

// actor: async let a = contador.sumarLento("A"); async let b = contador.sumarLento("B")
// cola:  contador.sumarLento("A"); contador.sumarLento("B")
```

Entorno: Swift 6.4 · Xcode 27.0 · modo Swift 6 · macOS 26

Pregunta 6 senior

Dos retiros en paralelo

La cuenta arranca con 100. Dos retiros de 80 corren en paralelo. ¿Qué imprime? ¿Hay data race?

```
actor CuentaBancaria {
    private(set) var saldo = 100

    func retirar(_ monto: Int) async -> Bool {
        guard saldo >= monto else { return false }
        await autorizar()                  // ej.: validar con un servicio externo
        saldo -= monto
        return true
    }

    private func autorizar() async {
        try? await Task.sleep(for: .milliseconds(10))
    }
}

let cuenta = CuentaBancaria()
async let r1 = cuenta.retirar(80)
async let r2 = cuenta.retirar(80)
let resultados = await [r1, r2]
print("retiros aprobados: \(resultados.filter { $0 }.count), saldo final: \(await
cuenta.saldo)")
```

Entorno: Swift 6.4 · Xcode 27.0 · modo Swift 6 · macOS 26

Pregunta 7 senior

Una continuation mal cerrada

Envolviste una API vieja de callbacks con `withCheckedThrowingContinuation`. ¿Qué pasa si el callback se llama dos veces? ¿Y si llega con `data` y `error` en `nil`?

```
func pedir(_ caso: String) async throws -> Data {
    try await withCheckedThrowingContinuation { continuation in
        pedirLegacy(caso) { data, error in
            if let data {
                continuation.resume(returning: data)
            } else if let error {
                continuation.resume(throwing: error)
            }
        }
    }
}

print("pido (\(caso))")
let data = try await pedir(caso)
print("llegaron \(data.count) bytes")
```

Entorno: Swift 6.4 · Xcode 27.0 · modo Swift 6 · macOS 26

Pregunta 8 senior

sync sobre colas seriales

¿Qué imprime cada caso? ¿Termina, se cuelga o crashea?

```
let otraCola = DispatchQueue(label: "recurso.serial")

// Caso 1: desde el main thread
print("antes")
DispatchQueue.main.sync { print("adentro") }
print("después")

// Caso 2: una cola serial que se hace sync a sí misma
let cola = DispatchQueue(label: "mi.serial")
cola.async {
    print("antes")
    cola.sync { print("adentro") }
    print("después")
}

// Caso 3: desde el main thread, sync a OTRA cola serial
print("antes")
otraCola.sync { print("adentro") }
print("después")
```

Entorno: Swift 6.4 · Xcode 27.0 · modo Swift 6 · macOS 26

Pregunta 9 senior

Un mock Sendable

El protocolo del servicio de red es `Sendable`. ¿Compila este mock en modo Swift 6? Y si le agregás `@unchecked Sendable`, ¿qué le estás prometiendo al compilador?

```
protocol ServicioDeRed: Sendable {
    func data(from url: URL) async throws → Data
}

final class MockServicioDeRed: ServicioDeRed {
    var dataADevolver = Data()
    var llamadas = 0

    func data(from url: URL) async throws → Data {
        llamadas += 1
        return dataADevolver
    }
}
```

Entorno: Swift 6.4 · Xcode 27.0 · modo Swift 6 · macOS 26 · Thread Sanitizer

Pregunta 10 senior

Un ViewModel que es actor

Para hacer thread-safe el ViewModel, alguien cambia `final class` por `actor`. En modo Swift 6, ¿compila? ¿Sirve?

```
import SwiftUI
import Combine

actor ContadorViewModel: ObservableObject {
    @Published var cuenta = 0
    func incrementar() { cuenta += 1 }
}

struct ContadorView: View {
    @StateObject private var vm = ContadorViewModel()

    var body: some View {
        Button("Cuenta: \(vm.cuenta)") {
            Task { await vm.incrementar() }
        }
    }
}
```

Entorno: Swift 6.4 · Xcode 27.0 · modo Swift 6 · macOS 26

Respuestas

1 Data(contentsOf:) en viewDidLoad

```
latido 1 a ~0 ms
latido 2 a ~100 ms
viewDidLoad: pido datos a ~200 ms
viewDidLoad: llegaron 12 bytes a ~1200 ms
latido 3 a ~1200 ms
latido 4 a ~1300 ms
...
```

Entre los 200 y los 1200 ms no hay ningún latido: el main thread estuvo bloqueado un segundo entero. En una app, eso es la UI congelada todo lo que tarde la red.

Por qué

`Data(contentsOf:)` es sincrónico: bloquea el hilo donde corre hasta que termina la descarga. Está pensado para archivos locales, no para red. Para red se usa `URLSession`, por ejemplo `let (data, _) = try await URLSession.shared.data(from: url)`, que suspende sin bloquear. Si agregás un `isLoading`, usá `defer { isLoading = false }` para que se resetee haya éxito o error.

> **Cómo decirlo en la entrevista**
>
> `Data(contentsOf:)` es sincrónico y está pensado para archivos locales. Para red uso `URLSession` con async/await, que suspende sin bloquear el main.

2 El callback de URLSession

```
antes de resume, main thread: true
en el callback, main thread: false
```

El completion de `URLSession.shared` llega en un hilo de background (la `delegateQueue` de la sesión), no en main. Tocar el label ahí es tocar UIKit fuera del main thread: bugs visuales intermitentes o crashes difíciles de reproducir.

Arreglo: `DispatchQueue.main.async { self.nombreLabel.text = ... }`, o pasar a `let (data, _) = try await URLSession.shared.data(from: url)` dentro de un contexto `@MainActor`.

Por qué

Quien decide el hilo del callback es la `delegateQueue` de la sesión. `URLSession.shared` usa una cola propia en background. La trampa extra: en modo Swift 6 el compilador no te salva del todo. Con un VC de AppKit, asignar el texto del label dentro del closure da solo un warning (las propiedades de los controles vienen de Objective-C con aislamiento "preconcurrency"); en modo Swift 5, ni eso.

> **Cómo decirlo en la entrevista**
>
> El completion de `URLSession.shared` llega en background. Lo que toque UI lo vuelvo a main, o uso la versión async desde un contexto `@MainActor`.

3 Task {} dentro de @MainActor

- `cargarPantalla()`: `1, 4, 2, 3`.
- `cargarPantallaInmediata()`: `1, 2, 4, 3`.

En ninguno de los dos casos la función espera al `await`: retorna en cuanto el Task se suspende (o antes).

Por qué

`Task { }` es un init sincrónico: encola el trabajo y vuelve al toque. Como hereda el main actor y el main actor está ocupado corriendo `cargarPantalla()`, el cuerpo del Task arranca recién cuando la función termina. El `await` de adentro es del Task nuevo, no de quien lo creó: si querés esperar, guardás el handle y hacés `await task.value`.

Desde Swift 6.2 hay un matiz: `Task.immediate` corre el cuerpo en el acto, de forma sincrónica, hasta el primer punto de suspensión. Por eso el `2` sale antes del `4`. Igual no bloquea al caller: en el primer `await` le devuelve el control.

> **Cómo decirlo en la entrevista**
>
> `Task {}` no espera: encola y retorna. Si necesito el resultado, guardo el handle y hago `await task.value`.

4 await en un ViewModel @MainActor

Con `fetch()` (solo `await`): la UI no se frena.

```
UI tick 1 (~0 ms)
fetch: arranca
UI tick 2 (~100 ms)
UI tick 3 (~200 ms)
fetch: termina
UI tick 4 (~300 ms)
```

Con trabajo sincrónico antes del `await`: el tick 2 llega a los 250 ms en vez de a los 100. La UI quedó congelada.

```
UI tick 1 (~0 ms)
fetch: arranca
UI tick 2 (~250 ms)
fetch: termina
UI tick 3 (~350 ms)
UI tick 4 (~450 ms)
```

Por qué

`await` suspende, no bloquea. El Task hereda el main actor, corre hasta el primer `await` y ahí suelta el hilo: el main actor queda libre para atender la UI hasta que el sleep (o la red) vuelva. Lo que sí congela es el trabajo sincrónico que corre en el main actor, haya o no un `await` después.

Ojo con "lo arreglo con `Task.detached`": si el trabajo pesado vive en un método `@MainActor`, el detached salta igual al main actor para llamarlo y bloquea igual. Solo sale del main si el trabajo pesado está fuera del main actor (por ejemplo, escrito en el closure del detached).

> **Cómo decirlo en la entrevista**
>
> `await` suspende, no bloquea. Lo que congela la UI es el trabajo sincrónico en el main actor, haya `await` después o no.

5 Actor contra cola serial

Cola serial: cada bloque corre entero, el contador termina en 2.

```
cola A: lee 0
cola A: escribe 1
cola B: lee 1
cola B: escribe 2
```

Actor: las dos llamadas leen 0 antes de que alguna escriba. El contador termina en 1 (se perdió una suma). El orden de las escrituras varía.

```
actor A: lee 0
actor B: lee 0
actor A: escribe 1     (o B primero)
actor B: escribe 1
```

Por qué

No hacen lo mismo en runtime. Los dos garantizan que nunca hay dos hilos tocando `valor` a la vez (no hay data race). Pero el actor es reentrante: en cada `await` suelta el aislamiento y deja entrar otra llamada. La cola serial ejecuta cada bloque de punta a punta. Con el actor, cualquier invariante que cruce un `await` se tiene que volver a chequear después del `await`.

> **Cómo decirlo en la entrevista**
>
> Un actor evita data races, pero es reentrante: en cada `await` puede entrar otra llamada. Una cola serial corre cada bloque entero.

6 Dos retiros en paralelo

Casi siempre:

```
retiros aprobados: 2, saldo final: -60
```

No hay data race: el actor nunca deja que dos hilos toquen `saldo` a la vez. Hay una race condition: el resultado depende del orden en que se intercalan las dos llamadas.

Por qué

El actor es reentrante. El primer retiro chequea `saldo ⩾ 80` (ve 100), llega al `await` y suelta el actor. Entra el segundo, también ve 100 y también pasa el `guard`. Después los dos restan. El leer-chequear-restar no es atómico porque lo corta un `await`. La memoria nunca se corrompe: el valor es incorrecto pero está bien definido (lost update).

Arreglo: no cruzar el `await` con un invariante a medias. O chequeás y restás sin `await` en el medio (después de autorizar), o reservás el monto antes del `await` y lo devolvés si falla.

```swift
func retirarSeguro(_ monto: Int) async -> Bool {
    await autorizar()
    guard saldo ⩾ monto else { return false }   // se chequea DESPUÉS del await
    saldo -= monto
    return true
}
```

> **Cómo decirlo en la entrevista**
>
> No es un data race, es una race condition por reentrancia. Chequeo el invariante después del `await`, o reservo el monto antes.

7 Una continuation mal cerrada

Dos veces: crash en el segundo `resume`.

```
pido (dos-veces)
_Concurrency/CheckedContinuation.swift:172: Fatal error: SWIFT TASK CONTINUATION MISUSE:
pedir(_:) tried to resume its continuation more than once, returning 1 bytes!
```

Nunca (los dos `nil`): no hay crash. El runtime avisa por consola y la task queda suspendida para siempre; `llegaron` nunca se imprime.

```
pido (nunca)
SWIFT TASK CONTINUATION MISUSE: pedir(_:) leaked its continuation without resuming it. This
may cause tasks waiting on it to remain suspended forever.
```

Por qué

El contrato es: `resume` exactamente una vez, por cualquier camino. La versión "checked" lo controla en runtime: el doble `resume` es un error de programador y mata el proceso; el `resume` que nunca llega se detecta cuando la continuation se libera, se loguea y la task queda colgada (sin timeout). El bug del ejemplo es el `if / else if` no exhaustivo: falta un `else` que haga `resume(throwing:)`.

Con `withUnsafeThrowingContinuation` no hay ese chequeo: el doble `resume` corrompe memoria (en la prueba abortó con un mensaje del allocator, no con uno que diga qué pasó) y el que nunca llega cuelga sin ningún aviso.

> **Cómo decirlo en la entrevista**
>
> `resume` va exactamente una vez, por cualquier camino. El doble `resume` crashea; si nunca llega, la task queda colgada para siempre.

8 sync sobre colas seriales

- Caso 1: imprime `antes` y crashea.
- Caso 2: imprime `antes` y crashea.
- Caso 3: imprime `antes`, `adentro`, `después`. Funciona.

Por qué

Una cola serial ejecuta un bloque por vez. En los casos 1 y 2, el hilo que está corriendo un bloque de esa cola se queda esperando un bloque nuevo que la misma cola nunca va a poder arrancar, porque primero tiene que terminar el actual: espera circular, deadlock. En Apple, libdispatch detecta ese caso y en vez de quedarse colgado mata el proceso (`EXC_BREAKPOINT`, "dispatch_sync called on queue already owned by current thread").

La condición del deadlock es self-sync (origen = destino), no "usar sync con una cola serial". El caso 3 es el patrón clásico para proteger un recurso compartido. Ojo: si la otra cola a su vez hace `sync` de vuelta a la primera, el deadlock vuelve.

> **Cómo decirlo en la entrevista**
>
> El deadlock aparece al hacer `sync` a la cola en la que ya estás. libdispatch lo detecta y crashea en vez de colgarse.

9 Un mock Sendable

No compila:

```
error: stored property 'dataADevolver' of 'Sendable'-conforming class 'MockServicioDeRed' is
mutable
```

Con `final class MockServicioDeRed: ServicioDeRed, @unchecked Sendable` compila. Lo que prometés es: "yo garantizo a mano que este estado mutable se accede de forma segura entre hilos". El compilador deja de chequear y te cree. Si no cumplís (como acá, `llamadas += 1` sin lock), tenés un data race real que nadie te va a marcar al compilar.

Por qué

`Sendable` en una clase exige que no tenga estado mutable, o que la sincronización la verifique el compilador. `@unchecked` apaga esa verificación, no agrega ninguna protección. Para un mock que solo se configura antes del test y se lee después, a veces alcanza; si el código bajo test lo llama en paralelo, cuenta mal. Opciones honestas: propiedades `let`, un `actor` como mock, o proteger el estado con `Mutex` (módulo Synchronization) o un lock, y recién ahí `@unchecked Sendable` dice la verdad.

> **Cómo decirlo en la entrevista**
>
> `@unchecked Sendable` no agrega protección, apaga el chequeo. Lo uso solo si el estado ya está protegido con `Mutex` o un lock.

10 Un ViewModel que es actor

Compila, con un warning (no error) en `body`:

```
warning: actor-isolated property 'cuenta' can not be referenced from the main actor
```

Y no sirve. `body` lee `vm.cuenta` de forma sincrónica desde el main actor, saltándose el aislamiento del actor: es exactamente el acceso concurrente que querías evitar. Además, `@Published` avisa `objectWillChange` desde el executor del actor, no desde main. Si quien escucha es código `@MainActor` (como una View o un VC), en Swift 6 el proceso crashea con la verificación de aislamiento en runtime.

La salida correcta para algo bindeado a la UI es `@MainActor final class` (o `@Observable @MainActor`). El `actor` va para servicios o stores que no se bindean directo a una View.

Por qué

Un actor sí puede conformar `ObservableObject` y tener `@Published`: el compilador lo acepta. El problema es que todo su estado queda aislado al actor, y SwiftUI necesita leerlo sincrónico en `body` y recibir los avisos en main. El compilador baja el error a warning porque el aislamiento al main actor de `body` viene de SwiftUI importado con concurrencia "preconcurrency"; en una función tuya marcada `@MainActor`, la misma lectura es error.

> **Cómo decirlo en la entrevista**
>
> Lo que se bindea a la UI va en `@MainActor`. El actor queda para servicios que ninguna View lee directo.

¿Cuántas acertaste?

Cada domingo te llega una pregunta así, con la respuesta verificada, y lo importante de la semana en Swift. Ya estás suscripto, no tenés que hacer nada.

Si te sirvió, pasáselo a alguien que esté preparando entrevistas: nicovma.github.io/newsletter